

EXPERIMENT 32: PROLOG PROGRAM TO IMPLEMENT PATTERN MATCHING

Aim

To write and execute a Prolog program to implement pattern matching using unification and backtracking.

Objective

- To understand pattern matching in Prolog.
- To understand unification.
- To compare patterns with available facts.
- To retrieve values matching a given pattern.

Algorithm

1. Define structured facts in the knowledge base.
2. Define a predicate for pattern matching.
3. Enter a query containing variables or fixed values.
4. Prolog attempts to unify the query with the available facts.
5. If a match is found, the corresponding values are returned.
6. If more than one match exists, Prolog uses backtracking.

Flowchart

START

|

v

Define Facts

|

v

Enter Pattern

|

v

Compare Pattern

with Facts

|

v

Match Found?

/\

Yes No

| |

v v

Display False

Match

|

v

More Matches?

/\

Yes No

| |

v v

Backtrack STOP

Code

% Pattern Matching

student(101, ravi, ai).

student(102, priya, ml).

student(103, arun, ai).

student(104, sneha, web).

match_student(ID, Name, Course) :-

student(ID, Name, Course).

Query 1

?- match_student(101, Name, Course).

Output

Name = ravi,

Course = ai.

Query 2

?- match_student(ID, Name, ai).

Output

ID = 101,

Name = ravi ;

ID = 103,

Name = arun ;

false.

Result

The Prolog program successfully performs pattern matching using unification and backtracking.

Conclusion

Thus, pattern matching was successfully implemented in Prolog using structured facts and predicates.

EXPERIMENT 34: PROLOG PROGRAM TO FIND THE NUMBER OF VOWELS

Aim

To write and execute a Prolog program to find the number of vowels present in a given string.

Objective

- To understand list processing in Prolog.
- To identify vowels from a string.
- To use recursion for processing characters.
- To count the total number of vowels.

Algorithm

1. Accept a string as input.
2. Convert the string into a list of characters.

3. Check the first character.
4. If the character is a vowel, increment the count.
5. If it is not a vowel, keep the count unchanged.
6. Recursively process the remaining characters.
7. Continue until all characters have been processed.
8. Display the total number of vowels.

Flowchart

START

|

v

Read String

|

v

Convert String

to Character List

|

v

Check Character

|

v

Is it a Vowel?

/ \

Yes No

| |

v v

Count Keep

+1 Count

\ /

v v

Process Next Character

|

v

Characters Remaining?

/\

Yes No

| |

v v

Repeat Display Count

|

v

STOP

Code

% Program to Count Vowels

vowel(a).

vowel(e).

vowel(i).

vowel(o).

vowel(u).

count_vowels([], 0).

count_vowels([H|T], Count) :-

vowel(H),

count_vowels(T, Rest),

Count is Rest + 1.

count_vowels([H|T], Count) :-

```
\+ vowel(H),  
count_vowels(T, Count).
```

```
find_vowels(String, Count) :-  
    string_chars(String, Characters),  
    count_vowels(Characters, Count).
```

Query

```
?- find_vowels("hello", Count).
```

Output

Count = 2.

Another query:

```
?- find_vowels("artificial intelligence", Count).
```

Output

Count = 9.

Result

The Prolog program successfully finds and displays the number of vowels present in a given string.

Conclusion

Thus, the number of vowels in a string was successfully determined using Prolog recursion, lists, and predicates.

```
1  % Pattern Matching  
2  
3  student(101, ravi, ai).  
4  student(102, priya, ml).  
5  student(103, arun, ai).  
6  student(104, sneha, web).  
7  
8  match_student(ID, Name, Course) :-  
9      student(ID, Name, Course).  
10
```

```
1 ?- consult('C:/Users/Varshini M/OneDrive/Desktop/Here!!!/ARTIFICIAL INTELLIGENCE/VSC EXPs/programs26.pl').  
true.
```

```
2 ?- match_student(101, Name, Course).  
Name = ravi,  
Course = ai.
```

```
3 ?- match_student(ID, Name, Course).  
ID = 101,  
Name = ravi,  
3 ?- match_student(ID, Name, Course).
```